Heart Disease Prediction using MLOps

MLOps Assignment-1

Aman Sagar
2024AC05987

Code Repository Link: https://github.com/aman-sagar-21/heart-disease-mlops
(the readme.md contains instructions for running and testing the application)

Overall Pipeline Video:
https://drive.google.com/file/d/1kgvRi_LPaWugzMWPrU340etUmyJtIx7p/view?usp=drive_link

Table of Contents

Contents

1. Introduction & Project Objectives

Machine Learning Operations (MLOps) combines machine learning, software engineering, and DevOps practices to automate the development, deployment, and maintenance of machine learning models. This project demonstrates an end-to-end MLOps workflow for heart disease prediction using the UCI Heart Disease dataset.

Project Objectives

- Perform exploratory data analysis.
- Clean and preprocess the dataset.
- Train and compare multiple machine learning models.
- Select the best-performing model.
- Track experiments using MLflow.
- Deploy the model with FastAPI.
- Containerize using Docker.
- Deploy using Kubernetes.
- Implement monitoring and logging.

A Brief Overview of the Workflow:

UCI Heart Disease Dataset --> Data Cleaning and Transformation --> Exploratory Data Analysis (EDA) --> Feature Engineering --> Model Training and Evaluation --> ML Flow Experiment Tracking --> Model Serialization (using Joblib, .pkl file) --> Rest API (Fast API) --> Docker Containerization --> Kubernetes deployment --> Monitoring and Logging

Code Repository Link: https://github.com/aman-sagar-21/heart-disease-mlops

Overall Pipeline Video:
https://drive.google.com/file/d/1kgvRi_LPaWugzMWPrU340etUmyJtIx7p/view?usp=drive_link

2. Dataset Description & Exploratory Data Analysis

2.1 Dataset Description

The Heart Disease dataset was obtained from the **UCI Machine Learning Repository**. It contains **303 patient records** with **13 clinical features** and one target variable indicating the presence of heart disease. The dataset includes demographic and clinical attributes such as age, blood pressure, cholesterol, chest pain type, and maximum heart rate.

Dataset Preview

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	1	145	233	1	2	150	0	2.3	3	0.0	6.0	0
1	67	1	4	160	286	0	2	108	1	1.5	2	3.0	3.0	2
2	67	1	4	120	229	0	2	129	1	2.6	2	2.0	7.0	1
3	37	1	3	130	250	0	0	187	0	3.5	3	0.0	3.0	0
4	41	0	2	130	204	0	2	172	0	1.4	1	0.0	3.0	0

2.2 Data Cleaning

The dataset was inspected for missing values, duplicate records, and incorrect data types. Missing values were identified in the **ca** and **thal** features and were imputed before model development. Data types were converted where necessary, and duplicate records were removed to improve data quality.

Missing Value Analysis

	Missing Values	Percentage
age	0	0.000000
sex	0	0.000000
cp	0	0.000000
trestbps	0	0.000000
chol	0	0.000000
fbs	0	0.000000
restecg	0	0.000000
thalach	0	0.000000
exang	0	0.000000
oldpeak	0	0.000000
slope	0	0.000000
ca	4	1.320132
thal	2	0.660066
target	0	0.000000

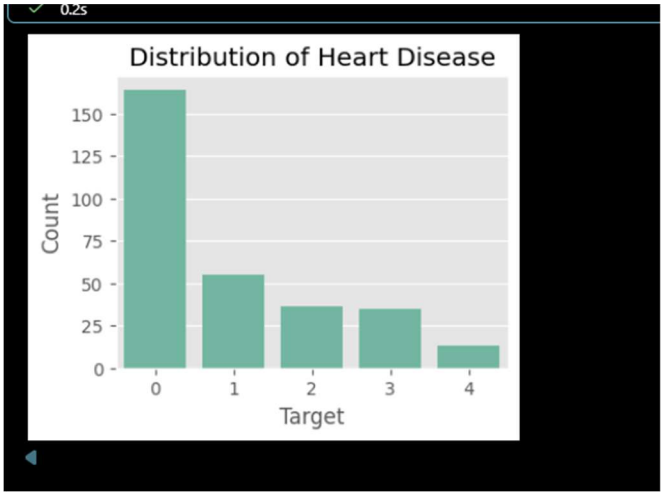
2.3 Exploratory Data Analysis

EDA was performed to understand feature distributions and relationships within the dataset.

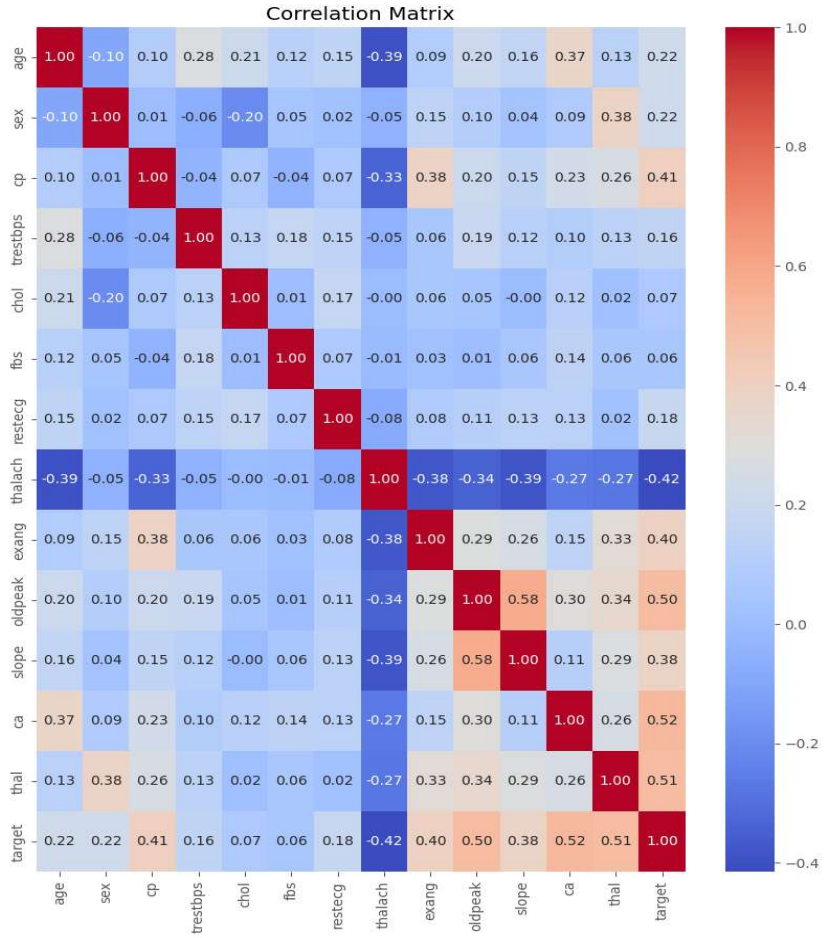
The following visualizations were generated:

- Target class distribution
- Histograms of numerical features
- Correlation heatmap
- Boxplots for outlier detection
- Pair plots for selected numerical features

Target Distribution



Correlation Heatmap



2.4 Key Observations

- The dataset contains **303 observations** and **13 input features**.
- Missing values were present only in **ca** and **thal**.
- Most numerical features showed reasonable distributions with a few outliers.
- Features such as **cp**, **ca**, **thal**, **oldpeak**, and **exang** showed stronger relationships with the target variable.
- The target classes were reasonably balanced, making the dataset suitable for classification.

3. Feature Engineering & Model Development

3.1 Feature Engineering

Feature engineering was performed to prepare the dataset for machine learning. The dataset was split into **80% training** and **20% testing** data. Missing values were handled using **SimpleImputer**, while numerical features were standardized using **StandardScaler**. A Scikit-learn **Pipeline** was implemented to integrate preprocessing and model training, ensuring a consistent workflow for both training and inference.

3.2 Model Development

Two supervised machine learning models were developed and evaluated:

- Logistic Regression
- Random Forest Classifier

Both models were trained using the same preprocessing pipeline to ensure a fair comparison. Performance was evaluated using **Accuracy, Precision, Recall, F1-score, ROC-AUC**, and **5-fold Cross-Validation**.

Table 1: Model Performance Comparison

Model Comparison

Metric	Logistic Regression	Random Forest
Accuracy	86.89%	**90.16%**
Precision	81.25%	**86.67%**
Recall	**92.86%**	**92.86%**
F1-Score	86.67%	**89.66%**
ROC-AUC	0.9513	**0.9578**

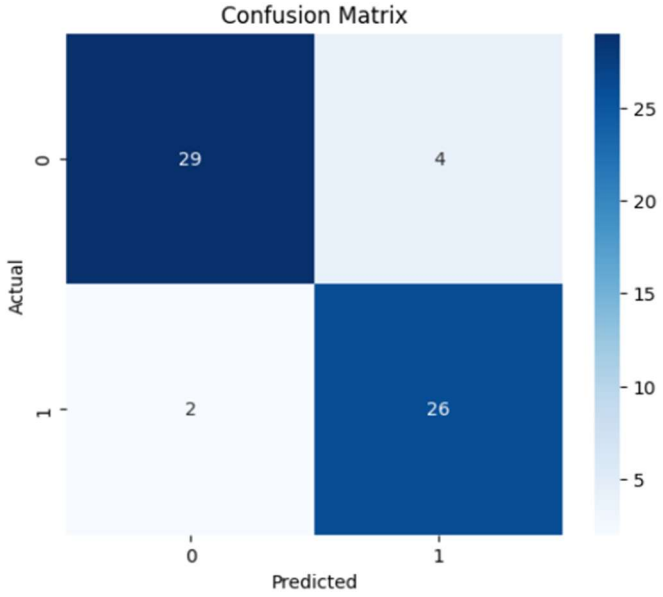
The Random Forest model was selected for deployment due to its superior predictive performance and consistent cross-validation results.

3.3 Model Selection

Among the evaluated models, the **Random Forest Classifier** achieved the best overall performance with higher Accuracy, F1-score, and ROC-AUC than Logistic Regression. It also produced a strong cross-validation score, indicating good generalization on unseen data.

Based on these results, the Random Forest model was selected as the final model for deployment.

Confusion Matrix (Random Forest)

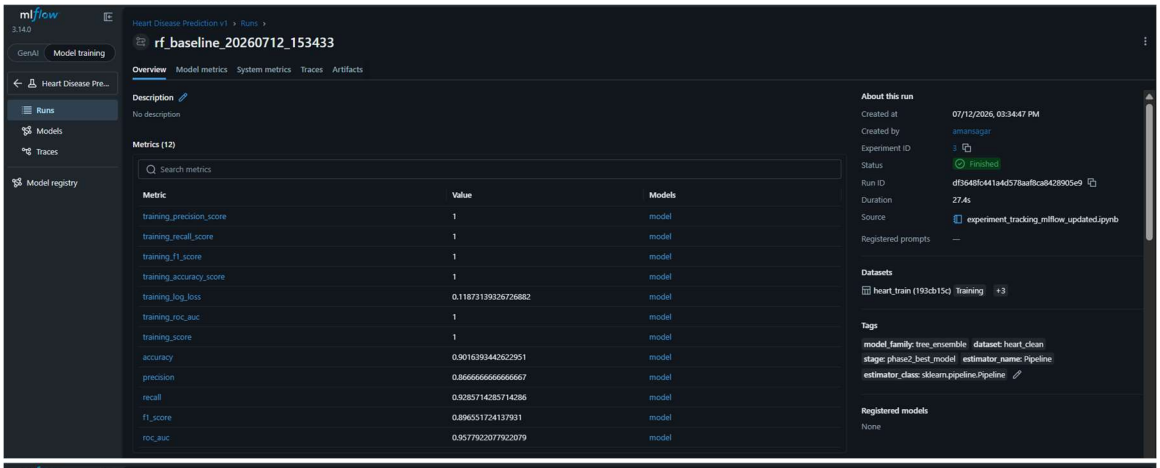


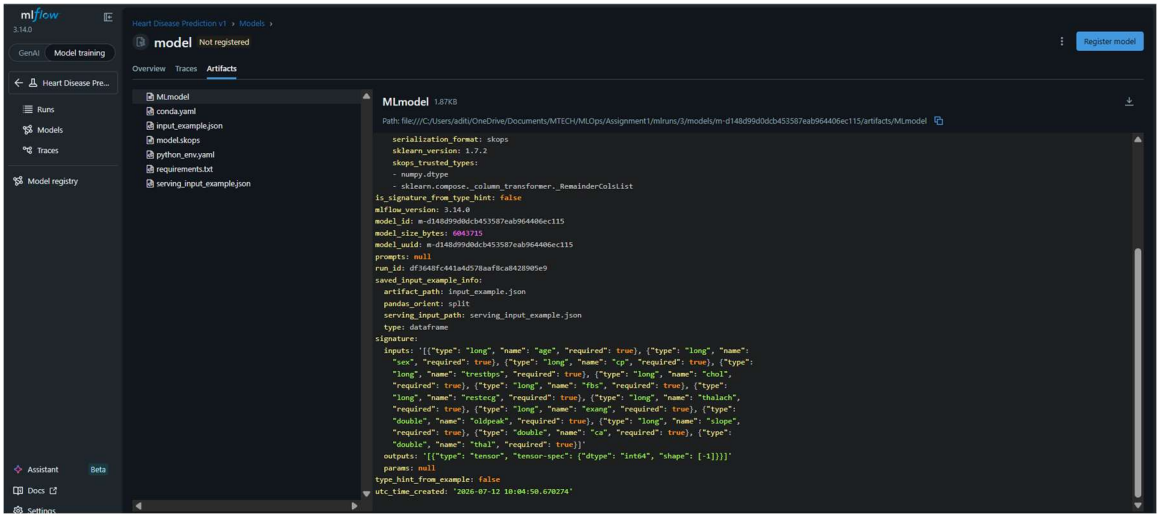
4. Experiment Tracking using MLflow

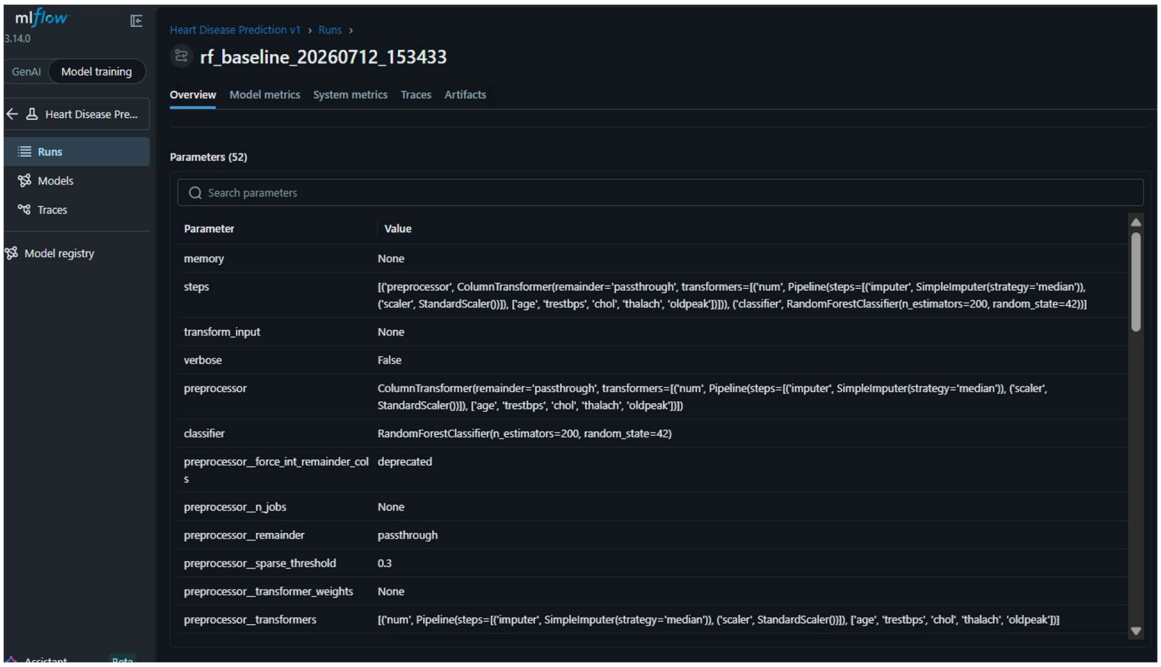
MLflow was integrated into the project to track and manage machine learning experiments. It was used to record model configurations, evaluation metrics, trained models, and artifacts, enabling reproducibility and easy comparison of different experiments.

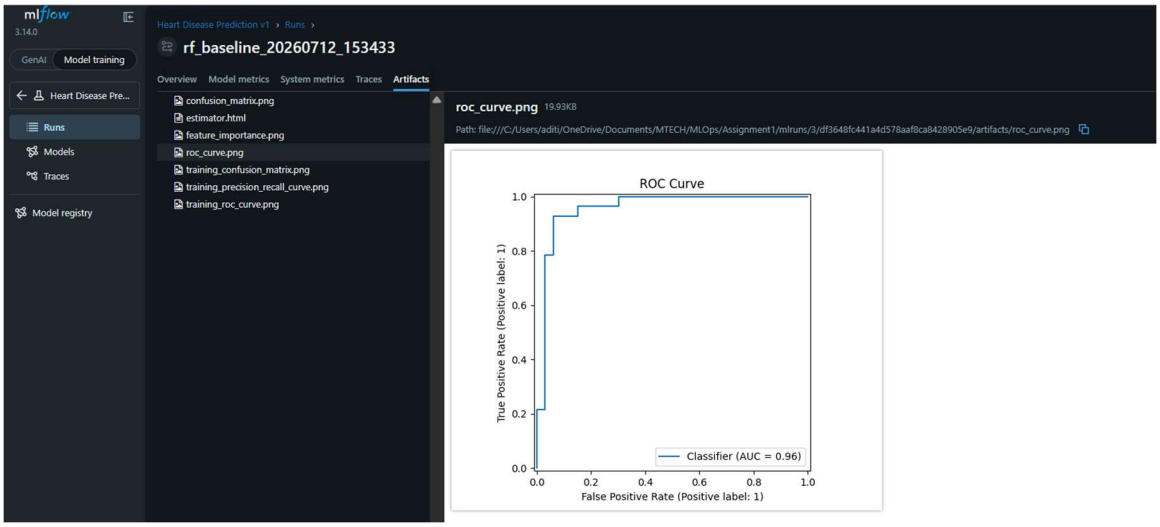
The following information was logged for each experiment:

- **Parameters:** Model type and training configuration.
- **Metrics:** Accuracy, Precision, Recall, F1-score, ROC-AUC, and Cross-Validation Score.
- **Artifacts:** Confusion Matrix, Classification Report, and trained model pipeline.
- **Model Registration:** The final Random Forest pipeline was logged and saved for deployment.









Summary

MLflow provided a centralized platform to compare model performance and maintain experiment history. Based on the recorded metrics, the **Random Forest Classifier** consistently outperformed Logistic Regression and was selected as the final model for deployment.

5. Model Packaging & FastAPI

5.1 Model Packaging

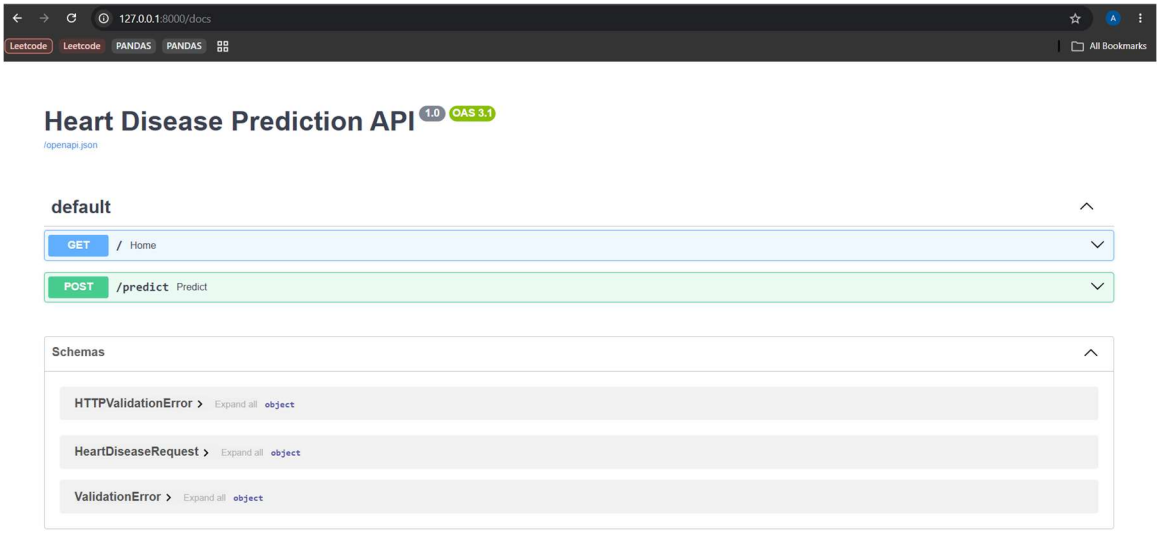
The final **Random Forest Pipeline**, including all preprocessing steps, was serialized using **Joblib** and saved as a reusable .pkl file. Packaging the complete pipeline ensures that the same preprocessing and prediction workflow is applied during both training and deployment, improving reproducibility and consistency.

5.2 FastAPI Implementation

A RESTful API was developed using **FastAPI** to serve the trained model. The API exposes the following endpoints:

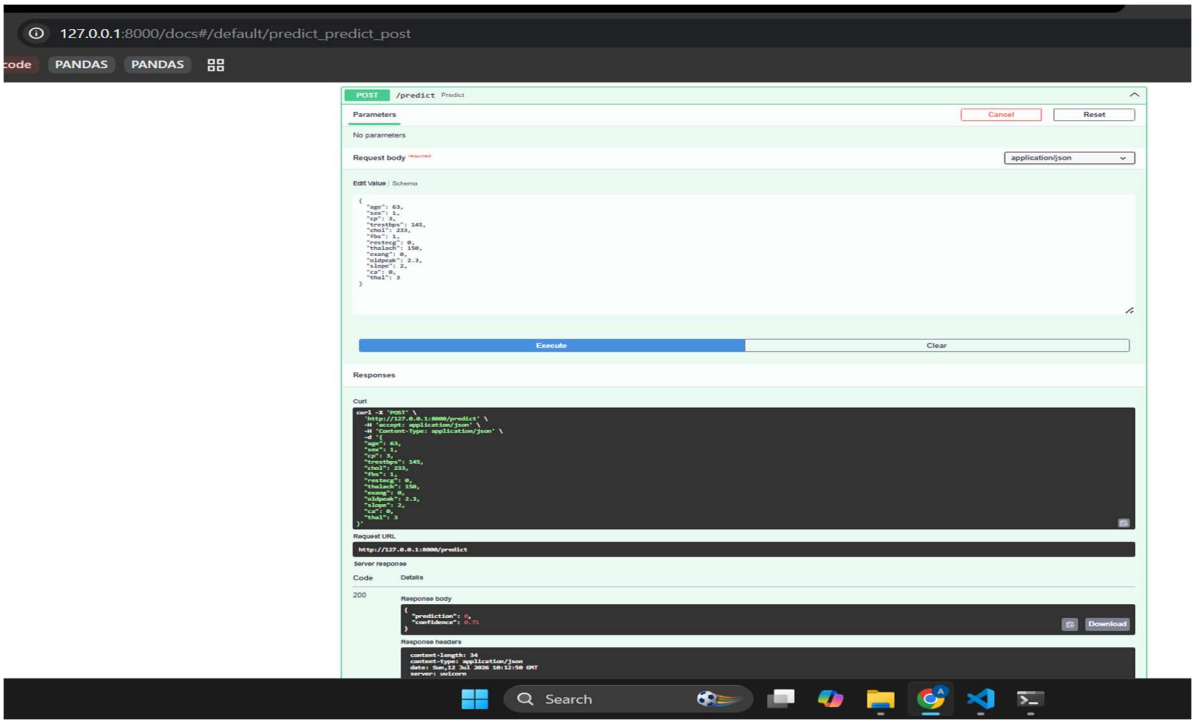
- **GET /** – Returns a welcome message.
- **POST /predict** – Accepts patient data in JSON format and returns the predicted heart disease status along with the prediction confidence.
- **GET /health** – Returns the application health status for monitoring purposes.

FastAPI Swagger Documentation



5.3 API Request & Response

The /predict endpoint accepts patient information as a JSON request containing all input features required by the trained model.



Request URL

```
http://localhost:8000/predict
```

Server response

Code	Details
200	Response body

```
{
  "prediction": 0,
  "confidence": 0.71
}
```

Response headers

```
content-length: 34
content-type: application/json
date: Sun,12 Jul 2026 11:15:54 GMT
server: uvicorn
```

Responses

6. CI/CD & Docker

6.1 Continuous Integration

A **GitHub Actions** workflow was implemented to automate the project's Continuous Integration (CI) process. The workflow is triggered on every push and pull request to the main branch and performs the following tasks:

- Installs project dependencies from requirements.txt.
- Executes unit tests using **Pytest**.
- Validates the project build to ensure the application is ready for deployment.

This automated pipeline helps identify issues early and ensures that only tested code is integrated into the project.

6.2 Docker Containerization

The FastAPI application was containerized using **Docker** to provide a consistent and portable deployment environment.

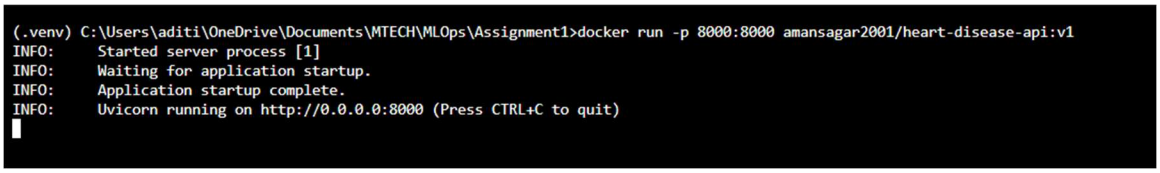
The containerization process includes:

- A **Dockerfile** defining the application environment and startup commands.
- A **requirements.txt** file specifying all required Python dependencies.
- Building the Docker image using docker build.
- Running the application in an isolated container using docker run.

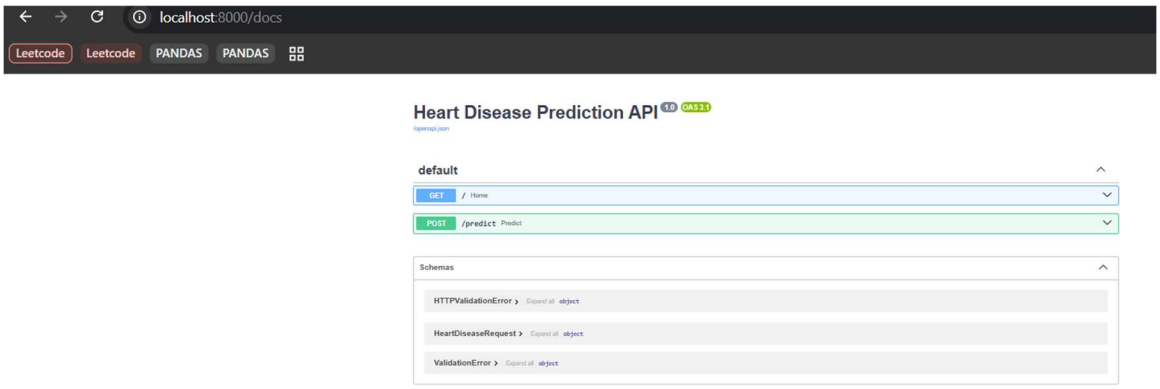
Containerization ensures that the application behaves consistently across different systems and simplifies deployment.

Docker Image Build

Running Docker Container



```
(.venv) C:\Users\aditi\OneDrive\Documents\MTECH\MLOps\Assignment1>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED              STATUS              PORTS                                       NAMES
d4c001ec8b26   amansagar2001/heart-disease-api:v1   "uvicorn app.main:ap…"   About a minute ago   Up About a minute   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   awesome_rubin
```



7. Kubernetes Deployment

7.1 Deployment

The Dockerized FastAPI application was deployed on a **local Kubernetes cluster** using **Docker Desktop Kubernetes**. A **Deployment YAML** file was used to define the application configuration, including the container image, resource limits, and **two replicas** for improved availability. A **Service YAML** file was created to expose the application within the cluster using a **LoadBalancer** service.

The application was accessed locally using **kubectl port-forward**, allowing requests to be forwarded from the local machine to the Kubernetes service.

7.2 Kubernetes Resources

The following Kubernetes resources were created and verified:

- **Deployment** – Manages the application pods and maintains two running replicas.

```
(.venv) C:\Users\aditi\OneDrive\Documents\MTECH\MLOps\Assignment1>kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
heart-disease-api   2/2     2            2           5h22m
```

- **Service** – Exposes the application and routes incoming requests to the pods.

```
(.venv) C:\Users\aditi\OneDrive\Documents\MTECH\MLOps\Assignment1>kubectl get services
NAME                    TYPE           CLUSTER-IP    EXTERNAL-IP   PORT(S)        AGE
heart-disease-service   LoadBalancer   10.96.98.76   172.18.0.5    80:32212/TCP   5h24m
kubernetes              ClusterIP      10.96.0.1     <none>        443/TCP        5h31m
```

- **Pods** – Run the containerized FastAPI application.

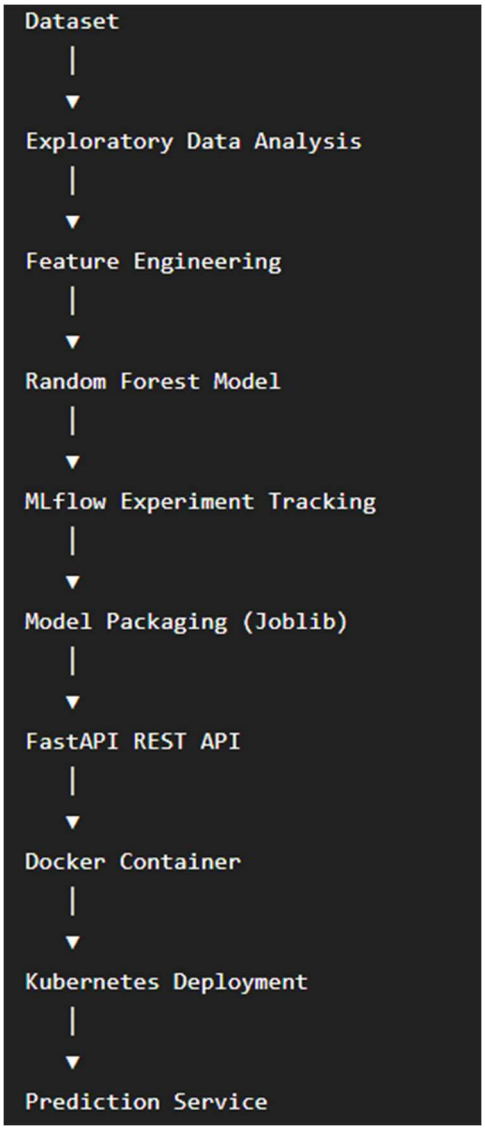
```
(.venv) C:\Users\aditi\OneDrive\Documents\MTECH\MLOps\Assignment1>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-api-74f9bdcd4b-fsnqd  1/1     Running   0          3h4m
heart-disease-api-74f9bdcd4b-ltlc7  1/1     Running   0          3h3m
```

Port Forwarding with Kubernetes

```
(.venv) C:\Users\aditi\OneDrive\Documents\MTECH\MLOps\Assignment1>kubectl port-forward service/heart-disease-service 8000:80
Forwarding from 127.0.0.1:8000 -> 8000
```

Aman Sagar (2 hours ago)

Architecture Overview:

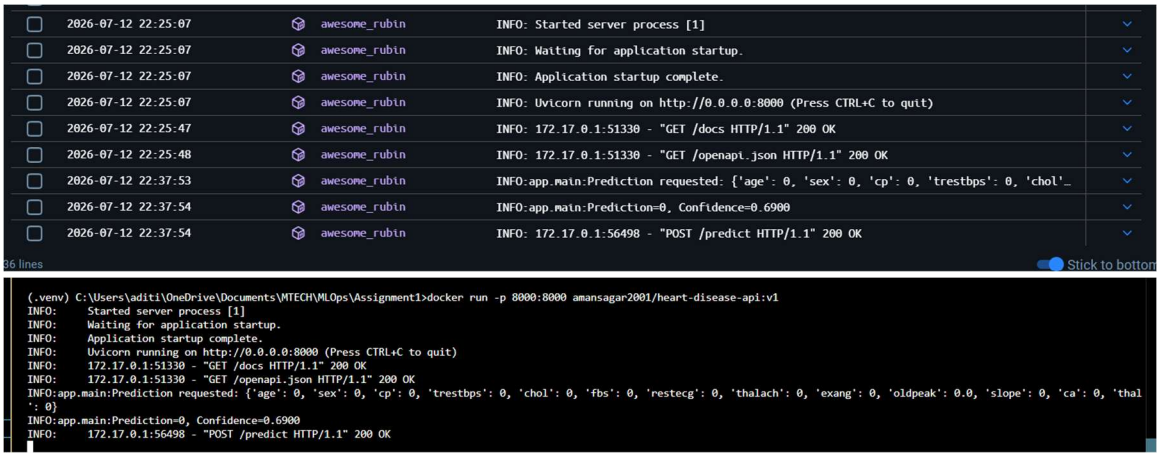


8. Monitoring, Challenges & Conclusion

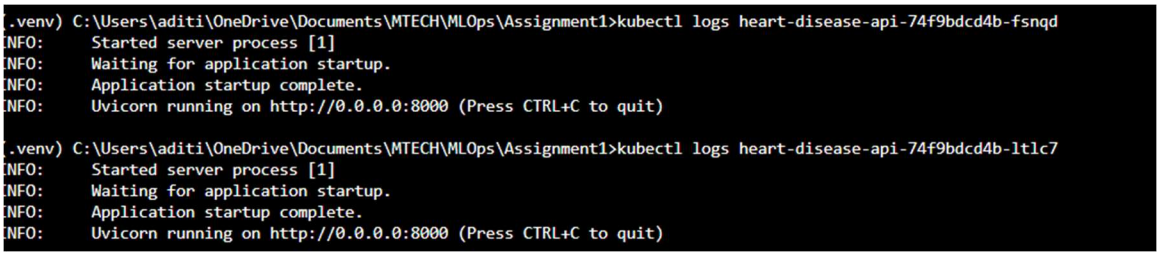
8.1 Monitoring

Basic monitoring was implemented using application logging and a health-check endpoint. The FastAPI application logs prediction requests, prediction results, and confidence scores, which can be viewed through **Docker** and **Kubernetes** logs. Additionally, a **/health** endpoint was implemented to verify that the application is running and ready to serve requests.

Docker Application Logs



Kubernetes Pod Logs



Kubernetes pod logs showing successful deployment of the FastAPI application.

8.2 CONCLUSION

This project successfully implemented an end-to-end MLOps pipeline for heart disease prediction using the UCI Heart Disease dataset. The workflow included data preprocessing, feature engineering, model training and evaluation, experiment tracking with MLflow, model packaging using Joblib, API development with FastAPI, containerization using Docker, deployment on Kubernetes, and basic monitoring through logging and health checks.

The project demonstrates how MLOps practices improve the reproducibility, scalability, and deployment of machine learning applications. By integrating machine learning with software engineering and DevOps tools, the developed solution provides a structured and production-oriented workflow that can be extended for real-world deployment.

References

1. UCI Machine Learning Repository. *Heart Disease Dataset*. Available at: https://archive.ics.uci.edu/dataset/45/heart+disease
2. Scikit-learn Developers. *Scikit-learn Documentation*. Available at: https://scikit-learn.org/stable/
3. MLflow. *MLflow Documentation*. Available at: https://mlflow.org/docs/latest/
4. FastAPI. *FastAPI Documentation*. Available at: https://fastapi.tiangolo.com/
5. Docker Inc. *Docker Documentation*. Available at: https://docs.docker.com/
6. Kubernetes Authors. *Kubernetes Documentation*. Available at: https://kubernetes.io/docs/
7. GitHub. *GitHub Actions Documentation*. Available at: https://docs.github.com/actions
8. Pandas Development Team. *Pandas Documentation*. Available at: https://pandas.pydata.org/docs/
9. NumPy Developers. *NumPy Documentation*. Available at: https://numpy.org/doc/
10. Joblib Developers. *Joblib Documentation*. Available at: https://joblib.readthedocs.io/